École Polytechnique

Computer Graphics

Assignment 1

Author:

Ioana Petan, École Polytechnique

Academic year 2025/2026

1 Lab 1: Introduction to rendering

1.1 Rendering a single sphere

The first scene consists of a single sphere placed on a black background. The light intensity I is set to 3×10^7, the image width W is set to 512, and the height H is also set to 512. The number of rays per pixel is 1, and the maximum number of light bounces is set to 5. The image took 98.2759 ms to render.

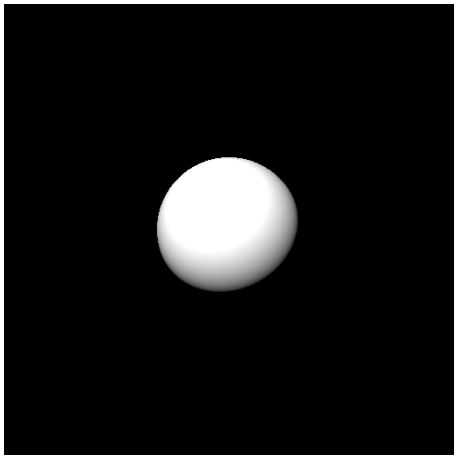

Figure 1: Rendering of a single diffuse sphere with direct lighting only.

1.2 Adding the surrounding walls

In the second step, we added walls to the scene in order to create a closed room around the sphere. The scene now contains the sphere with the left wall, right wall, back wall, front wall, ceiling, and floor. The sphere itself remains diffuse in this render, without mirror reflection.

The light intensity I remains set to 3×10^7, the image width W is 512, and the height H is also 512. The number of rays per pixel is 1, and the maximum number of light bounces is set to 5. There is no indirect lighting or antialiasing.

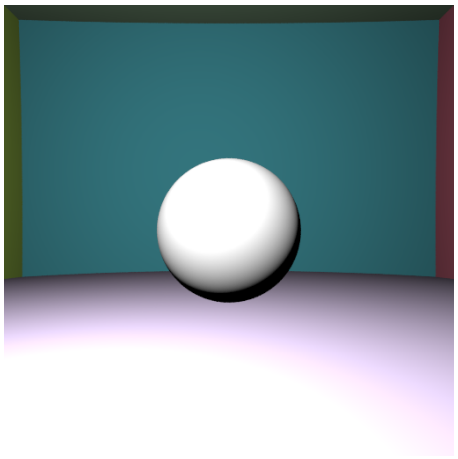

Figure 2: Rendering of the central sphere after adding the surrounding walls.

1.3 Adding mirror properties

In the next render, mirror properties are enabled for the sphere. This allows rays hitting the sphere to be recursively reflected in the mirror direction, producing reflections of the surrounding colored walls on its surface. The walls remain present in the scene, but the main difference from the previous render is the reflective behavior of the sphere.

The light intensity I remains set to 3×10^7, the image width W is 512, and the height H is also 512. The number of rays per pixel is 1, and the maximum number of light bounces is set to 5. There is no indirect lighting or antialiasing. The image took 343.165 ms to render.

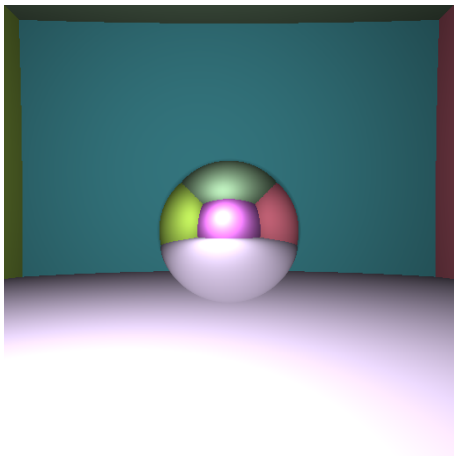

Figure 3: Rendering of the scene with surrounding walls and mirror properties enabled for the central sphere.

1.4 Adding shadows

In this third render, shadows are added to the scene. To determine whether a point is illuminated, a shadow ray is sent from the surface point toward the light source. When another object blocks this path, the point is rendered in shadow. The image took 579.17 ms to render.

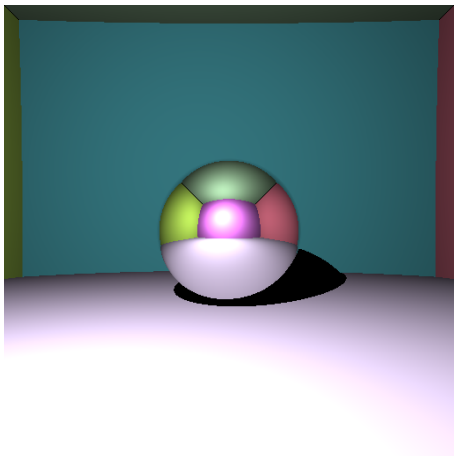

Figure 4: Rendering of the scene with walls, mirror properties, and shadows enabled.

2 Lab 2: Indirect lighting and antialiasing

2.1 Indirect lighting

In the first step of Lab 2, indirect lighting is added to the scene while keeping only one primary ray per pixel. Along the direct contribution from the point light source, we generate random secondary rays around the surface normal according to a cosine-weighted distribution. These rays are used to estimate the indirect light contribution coming from the surrounding scene.

The image width is set to $W = 512$, and the height is set to $H = 512$. The light intensity remains $I = 3 \times 10^7$, gamma is applied with $\gamma = 2.2$, and the maximum number of light bounces is set to 5. Since antialiasing is not yet applied, only one primary camera ray is cast per pixel. The image took **Render time: 2688.98 ms** to render.

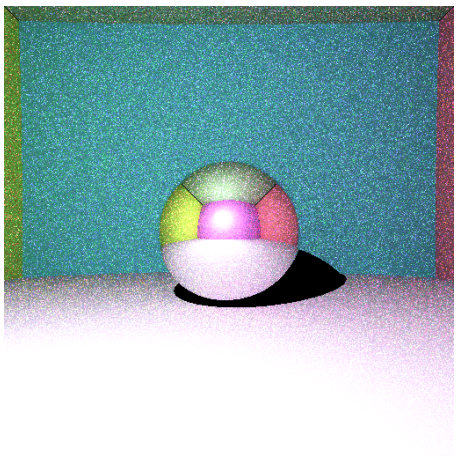

Figure 5: Rendering with indirect lighting, but without antialiasing.

2.2 Adding antialiasing

In the second step, we add antialiasing to the indirect lighting render. Instead of casting only one ray through the center of each pixel, several slightly perturbed rays are generated and averaged. This produces smoother object contours and scene boundaries.

The image width and height remain 512, the light intensity is still $I = 3 \times 10^7$, and the maximum number of light bounces remains set to 5. In this render, 32 rays per pixel are used for antialiasing and averaging. The image took **Render time: 552.183 ms** to render.

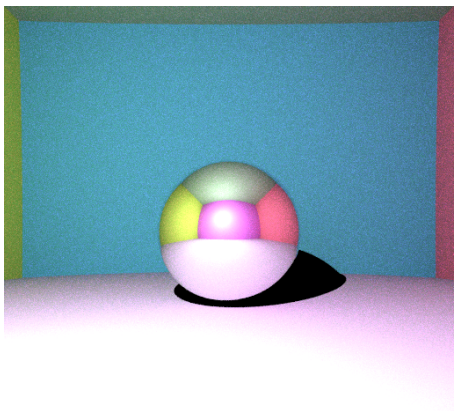

Figure 6: Rendering with indirect lighting and antialiasing.

3 Lab 3: Meshes and textures

3.1 Rendering a triangle mesh

In the third lab, we add triangle meshes to the ray tracer. The cat model is loaded from an `.obj` file and rendered inside the scene with the colored walls. To determine whether a ray intersects the mesh, we apply the Möller–Trumbore ray-triangle intersection algorithm to all triangles of the model.

To accelerate the computation, we build a bounding box around the entire mesh. Before testing intersections with the individual triangles, each ray is first tested against this bounding box. Rays that do not intersect it are ignored, reducing the number of unnecessary ray-triangle intersection computations.

For this render, the image width and height are both set to 512. The light intensity is $I = 3 \times 10^7$, the gamma correction value is 2.2, and the maximum number of light bounces is set to 2. The final image is computed using 32 rays per pixel, which also includes antialiasing and the indirect lighting

contribution introduced in Lab 2. The cat mesh contains **Number of triangles: 3954** triangles. The image took 448706 ms, or approximately 448.7 s, to render.

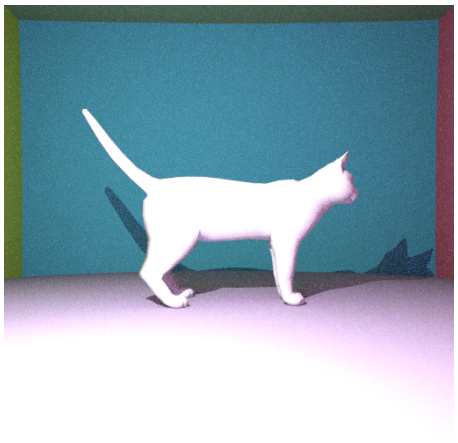

Figure 7: Rendering of the cat mesh using Möller–Trumbore ray-triangle intersections accelerated by a single bounding box.

4 Comments

The code was developed across the different lab sessions. Each new feature was added on top of the previous implementation: sphere-ray intersections, walls, mirror reflections, shadows, indirect lighting, antialiasing, and finally triangle mesh rendering with bounding-box acceleration.

During the lab sessions, I also exchanged ideas and discussed parts of the implementation with Klyuchereva Sofya, Mateo Fatas, Sacha Gregoire, and Dervishi Joni.